



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING

Descriptive Written Exam

Name :- M.VIGNESH

Register Number :- 192521020

COURSE NAME :Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3

3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3

8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3

15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I M.VIGNESH (192521020) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

BSPNAIDU.//

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Testing as an Engineering Activity

Testing a banking application should follow an engineering-based method with planned activities, measurable objectives, and controlled documentation.

- First, study the requirements and identify high-risk functions such as login, balance enquiry, fund transfer, and payment processing.
- Prepare a test plan defining scope, responsibilities, schedule, resources, and risks.
- Design test cases using equivalence partitioning, boundary value analysis, decision tables, and state-transition techniques.
- Establish expected results before execution so that failures can be recognized objectively.
- Use test-management and defect-tracking tools to maintain traceability.
- Apply functional, integration, security, performance, and regression testing according to risk.
- Replace random testing with risk-based test selection and reusable test suites.
- For example, fund transfer should cover valid accounts, insufficient balance, invalid beneficiaries,
- Limits, duplicate requests, and network interruption.
- After corrections, perform retesting followed by regression testing.
- This engineering discipline strengthens overall quality and reduces avoidable defects.

2. Structured Testing Process for ATM

A reliable ATM requires a testing process that begins during requirements and continues until release.

- During requirement analysis, identify rules for PIN validation, account balance,
- Withdrawal limits, cash availability, and transaction completion.

- In test planning, define scope, responsibilities, environments, resources, risks, and schedules.
- During test design, create positive, negative, boundary, and security cases.
- During execution, compare actual behavior with predefined expected results and record deviations.
- Every defect should be documented with severity, priority, evidence, and reproduction steps.
- After developers correct an issue,
- Testers perform retesting to confirm the fix and regression testing to ensure existing functions remain stable.
- ATM scenarios should include correct PIN, wrong PIN, insufficient funds, invalid amounts,
- Daily limits, unavailable cash, and interrupted communication.
- This process improves consistency, traceability, reliability, and customer confidence significantly.

3. Error, Defect and Failure

In software testing, an error, defect, and failure describe different stages of a problem.

- An error is a human mistake made while interpreting requirements, designing a solution, writing code,
- Configuring a system, or preparing tests.
- A defect is the incorrect software element introduced because of that mistake.
- A failure is the visible incorrect behavior that occurs when the defective software is executed under a particular condition.
- For example, suppose a developer misunderstands a transaction limit and writes an incorrect validation rule.
- The misunderstanding is the error, the wrong rule in the program is the defect,
- And accepting an unauthorized transaction during execution is the failure.
- Clear defect reports should distinguish these concepts and provide reproduction steps, expected behavior, actual behavior,

- Precise terminology therefore makes defect communication more accurate, actionable,
- And efficient throughout the complete software project lifecycle.

4. Defect Clustering and Test Prioritization

Defect clustering means that defects are often concentrated in a small number of modules rather than being distributed evenly throughout a system.

- This principle should influence test planning because limited testing time must be directed toward areas with greater risk.
- Historical defect records can reveal modules that repeatedly fail, while complexity,
- Recent changes,
- Business importance, and security impact provide additional prioritization factors.
- Test cases for high-risk modules should be executed earlier and more thoroughly.
- For example, in an online shopping system, payment,
- Checkout,
- And order confirmation deserve more attention than a basic profile display.
- Recently modified payment code should also receive strong regression testing.
- Low-risk features can be scheduled after critical functions have been covered.
- This evidence-based prioritization helps teams concentrate effort on modules most likely to cause serious failures in production.

5. Defect Classification and Defect Repository

Defect classification helps a testing team understand where problems occur and what type of failure they represent.

- Defects may originate in requirements, design, coding, integration, configuration, or data handling.
- They can also be grouped as functional, performance, security, usability, interface, compatibility, or data defects.

- A defect repository should capture an identifier, description, module, environment, severity,
- Priority, status, root cause, and resolution.
- When repository information is analyzed across releases, recurring patterns become visible.
- For example,
- Repeated interface defects in a payment module may indicate weak design review or insufficient compatibility testing.
- Testers can convert important historical defects into regression cases and use root-cause analysis to identify preventive actions.
- Code reviews, improved requirements,
- Automated regression suites, and focused exploratory testing can then address the causes.
- This approach supports prevention as well as detection.

6. Mobile Banking Fund Transfer

Frequent crashes during mobile banking fund transfers require a systematic investigation rather than repeated random execution.

- The tester should first reproduce the crash and collect logs, transaction details, application version,
- Device information, operating-system version, and network conditions.
- Normal transfers should be tested along with invalid account numbers, insufficient balance,
- Excessive amounts, transaction limits,
- Duplicate requests, session expiry, and network interruption.
- Different devices and operating systems should also be considered because environment-specific problems may trigger failures.
- Integration testing must verify communication among the mobile application, database,
- Authentication service, and banking transaction service.

- After the development team fixes the root cause, retesting confirms the correction,
- While regression testing checks related functions.
- Improves stability.

7. Testing and Debugging

Testing and debugging support software quality, but they are not the same activity.

- Testing is performed to reveal defects, evaluate behavior, and determine whether requirements are satisfied.
- Debugging begins after a problem has been identified and focuses on locating the underlying cause and correcting the implementation.
- For example,
- A test may show that an account balance becomes incorrect after a transfer.
- The tester reports the failure with evidence, while the developer examines calculations,
- Database updates, or service responses to locate the cause.
- Once the code is corrected,
- Testers perform retesting to verify the specific fix and regression testing to ensure other features were not damaged.
- Confusing the two activities can lead to incomplete reports, unclear responsibilities,
- And wasted effort.
- This separation keeps responsibilities clear and prevents duplicated effort unnecessarily.

8. Cross-Browser Testing

Cross-browser testing is necessary because the same web application may behave differently across browser engines,

- Versions,
- And operating systems.
- The test strategy should identify supported combinations and create a browser compatibility matrix.
- Important functions such as login, registration, forms, navigation,
- Search, payments, and transaction confirmation should be tested on each relevant combination.
- Visual checks should cover layout, fonts, buttons, menus, responsive behavior, and screen sizes.
- Functional differences can arise from JavaScript execution, CSS interpretation,
- Browser settings, or unsupported features.
- Therefore, compatibility testing should be combined with functional and responsive testing.
- Repetitive scenarios can be automated with tools such as Selenium to improve coverage and reduce execution time.
- These practices increase confidence that important functions work correctly across supported browser environments.

9. Payroll Logical Errors

Logical defects in a payroll system are especially serious because the program may execute normally while producing incorrect salary results.

- Such problems can involve basic calculations, overtime rules, tax deductions, bonuses,
- Allowances, employee categories, or business policies.
- Defects should be classified according to their cause, such as calculation, business-rule,
- Data, or requirement problems.
- Testers should verify normal salaries as well as boundary conditions around minimum and maximum values.
- Different employee types and combinations of overtime, deductions, tax, and bonuses should be included.
- Equivalence partitioning can group similar salary inputs,

- While boundary value analysis checks values around important limits.
- Unit tests can identify calculation errors before integration,
- And code reviews can expose incorrect business logic.
- This reduces payroll calculation risk.

10. Structured Test Case Design

Random testing alone cannot provide dependable coverage because important requirements and risk conditions may remain untested.

- A structured test-case design process should begin with requirements, business rules, and identified risks.
- Each test case should define an identifier, objective, preconditions, inputs,
- Execution steps, expected result, actual result, and status.
- Equivalence partitioning divides inputs into meaningful valid and invalid groups,
- While boundary value analysis focuses on values near limits.
- Decision tables are useful when several business conditions determine an outcome,
- And state-transition testing suits systems whose behavior changes according to state.
- Risk-based testing gives earlier attention to critical functions.
- For example, a banking transaction limit should be checked below the limit,
- Exactly at the limit, and above it.
- This strengthens useful test coverage.

11. Healthcare Data Inconsistency

Data inconsistency in a healthcare application can create serious problems because different modules may display or store conflicting patient information.

- Testing should begin by identifying where shared data is created, modified,
- Transferred, and displayed.
- Patient details should be compared across registration, medical records, billing,
- Laboratory information, and reporting modules.
- Database testing should verify correct insertion, update, retrieval, and persistence of records.
- Integration testing should confirm that information passed between modules is complete and accurate.
- Valid, invalid, duplicate, missing, and boundary inputs should also be exercised.
- Transaction tests should check whether updates are committed correctly and whether failures leave data in a safe state.
- Each defect report should include the affected module, data values, expected result,
- Actual result, environment, and severity.
- These controls protect consistency across healthcare workflows and records.

12. Defect Repository and Repeated Defects

A defect repository becomes valuable when a company uses it not merely to store issues,

- But to learn from them.
- Each record should contain the defect identifier, module, description, type, severity,
- Priority, release, root cause, status, and resolution.
- Historical information can be grouped by module, release, defect category, and frequency to reveal repeated patterns.
- If the same problem returns after several releases,
- The original cause may not have been adequately corrected.
- Testers should analyze recurring defects and convert significant cases into permanent regression tests.

- Modules with high failure rates should receive additional reviews, focused testing, and stronger automation.
- Root-cause analysis can reveal weaknesses in requirements, design, coding, or review practices.
- Automated regression suites can then verify that previous failures do not return.
- This makes defect history actionable for testers.

13. Peak-Usage Failure in Learning System

A learning platform that fails during peak usage indicates that normal functional testing was not enough to evaluate real workload conditions.

- Test planning should identify expected user volumes, peak demand,
- Critical workflows, and performance objectives before execution.
- Load testing can simulate the expected number of concurrent users,
- While stress testing pushes the system beyond its normal capacity to reveal breaking points.
- Response time, throughput, server resources, database behavior, and error rates should be measured.
- Critical activities such as login, course access, examination,
- File submission, and result viewing should be executed concurrently.
- Database connections, application servers,
- And network services should also be monitored because bottlenecks may occur outside the user interface.
- For example, thousands of students accessing an examination simultaneously should be simulated before deployment.
- This improves availability.

14. Early Tester Involvement in SDLC

Testers should participate throughout the SDLC instead of joining only after development is completed.

- During requirements analysis, they can identify ambiguous statements, missing acceptance conditions,
- Security needs, and unrealistic assumptions.
- During design, testers can review architecture, interfaces, risks, and testability.
- While development is progressing, they can prepare test cases,
- Support integration checks, and help define automation opportunities.
- Early involvement allows defects to be prevented before they become expensive implementation problems.
- For example, in a banking project,
- Testers can review transaction rules and security requirements before coding begins.
- Regular collaboration among testers, developers, business analysts, and stakeholders improves shared understanding.
- Reviews can identify incorrect requirements before they reach implementation, reducing rework later.
- Continuous involvement makes quality responsibilities visible and encourages earlier risk identification across the project development lifecycle.

15. Incomplete Test Coverage

Incomplete test coverage occurs when important requirements, inputs, conditions,

- Paths, or interactions are left untested.
- To improve coverage,
- Testers should map test cases to functional and non-functional requirements and prioritize areas according to risk.
- Positive and negative scenarios must both be represented.
- Equivalence partitioning can cover groups of similar inputs,
- While boundary value analysis targets limits where failures commonly occur.
- Edge cases, invalid values, missing information, and unusual combinations should also be included.

- For an ATM withdrawal feature, useful cases include a valid amount, zero, negative input,
- Maximum permitted amount, amount above the limit, and insufficient balance.
- Integration and security conditions should be considered where applicable.
- This strengthens overall release confidence for users everywhere.

16. Defect Reporting

Effective defect reporting allows developers to understand a problem quickly and reproduce it without unnecessary investigation.

- Every report should have a unique identifier, meaningful title,
- Affected module,
- Environment, application version, and clear reproduction steps.
- The expected result must be separated from the actual result so the deviation is immediately visible.
- Severity and priority should be assigned consistently, and screenshots,
- Logs,
- Videos,
- Or other evidence should be attached when useful.
- For example,
- Stating that a transaction fails when the amount exceeds the permitted limit is more informative than simply writing transaction problem.
- A centralized tracking system should be used so that ownership and status remain visible.
- Clear reports speed defect resolution.

17. Process-Based Testing

Process-based testing establishes a common method so that different testers and modules are handled consistently.

- Test design, execution,
- Defect reporting, retesting, regression testing, and closure.
- Every tester should use the same test-case structure and common definitions for severity and priority.
- Test plans and cases should be reviewed before execution to identify missing scenarios and unclear expectations.
- Checklists can prevent important activities from being forgotten, while metrics can monitor progress, defect trends,
- Coverage, and execution status.
- For example,
- All modules in a banking application can follow one defect-reporting and verification procedure even when different teams own those modules.
- Standardization improves traceability because activities are recorded in a predictable manner.
- Standardization supports predictable testing outcomes and easier coordination between teams consistently.

18. Boundary Testing and Edge Cases

Boundary testing focuses on values at the edges of an allowed range because defects frequently occur near limits.

- Edge cases may be missed when testers concentrate only on ordinary inputs.
- Boundary Value Analysis requires testing just below a limit, exactly at the limit,
- And just above it.
- For an application accepting ages from 18 through 60, suitable values include 17, 18, 19, 59, 60, and 61.
- The same principle applies to transaction amounts, file sizes, quantities,
- Dates, account limits, and character lengths.
- Minimum and maximum values should be identified from requirements before test execution.

- These scenarios should be included during test-case design rather than added only after a production failure.
- Boundary testing improves coverage and exposes validation mistakes that normal inputs may not reveal.
- These checks expose defects.

19. Defect Documentation and Repository

A well-maintained defect repository provides a central record for controlling, analyzing,

- And learning from software problems.
- Each defect should have a unique identifier, clear description, affected module,
- Environment, reproduction steps,
- Expected result, actual result, severity, priority, and status.
- Root cause and resolution details should also be preserved so that future teams understand what happened.
- A defect can move through states such as New, Assigned,
- Fixed,
- Retested, and Closed, providing visibility into its lifecycle.
- Historical records help identify modules with frequent failures and reveal repeated causes across releases.
- Important previous defects can be converted into regression tests so that the same problem is checked automatically in future versions.
- This preserves useful defect knowledge for future releases.

20. Banking Transaction Processing

Banking transaction processing needs systematic testing because incorrect transactions can affect money, customer records,

- Testing should begin with requirements and risk identification, followed by positive and negative scenarios.
- Successful transactions, insufficient balances, invalid account details, transaction limits,
- Duplicate requests, timeouts, and network interruptions should all be exercised.
- Boundary value analysis can verify limits, while equivalence partitioning organizes valid and invalid input groups.
- Decision tables help evaluate combinations of transaction conditions,
- And state-transition testing can check stages such as initiated,
- Authorized,
- Completed, failed, or reversed.
- Integration testing should verify communication among the banking interface, database, authentication service, and transaction service.
- Security testing must cover authentication, authorization, and protection of sensitive operations.
- This approach reduces financial transaction risk for customers.